

سوالات تحلیلی

سوال ۱: هنگام راه اندازی (ریست)، پردازنده ARM چگونه و از کجا شروع به اجرای کد میکند؟

در کد Arm یک Area به اسم RESET که در واقع نقش یک Vector Table را برای شناسایی آدرس شروع استک، reset handler و اکسپشن ها و اینتراپت ها است، وجود دارد.

هر گاه پردازنده شروع به کار می کند، در ابتدا entry که مربوط به Stack Pointer است، را ست می کند. پس از آن به سراغ entry دوم (که همان محل Reset_Handler است و آن را export می کند) می رود و آن را در

PC (Program Counter) کپی می کند و روند اجرای کد (execution) را از این آدرس آغاز می کند.

بنابراین کد برنامه در واقع از جایی که برچسب (label) ریست هندلر (Reset_Handler) توسط برنامه نویس مشخص شده است، شروع می شود.

```

AREA    RESET, DATA, READONLY
EXPORT  __Vectors
EXPORT  __Vectors_End
EXPORT  __Vectors_Size

__Vectors DCD __initial_sp      ; Top of Stack
          DCD Reset_Handler    ; Reset Handler
  
```

```

Reset_Handler
; *** MAIN STARTS HERE ***
    LDR R0, =COEFS ; ADDRESS OF COEFS
    BL KHPA
ENDLESS B ENDLESS ; END OF MAIN
; *****
  
```

سوال ۲: فایل scr که هنگام ایجاد یک پروژه Keil در پوشه پروژه ساخته می شود چه محتوایی دارد و چه کاربردی دارد؟

فایل scr، در واقع فایل Linker scatter loader است. که توسط لینکر تولید می شود و می توان آن را دستی طبق نیاز خود، تغییر داد. در این فایل layout و مکان بخش های code و data در مموری را مشخص می کنیم. برای مثال یکی از کاربرد های تغییر آن می تواند بهبود استفاده از حافظه (مموری) توسط برنامه با کنترل و ست کردن alignment ها و padding و ترتیب قسمت (section) های ورودی باشد. محتوای آن به طور پیش فرض به صورت ذیل می باشد:

سوال ۳: پس از اجرای دستورات زیر داده ها چگونه در حافظه ذخیره میشوند؟

```
LDR    R2,=0xFA98E322
```

```
LDR    R1,=0x20000100
```

```
STR    [R1],R2
```

الف) با فرض Little-Endian:

0x20000100	0x20000108	0x20000110	0x20000118	0x20000120
22	E3	98	FA	

ب) با فرض Big-Endian:

0x20000100	0x20000108	0x20000110	0x20000118	0x20000120
FA	98	E3	22	

گزارش دستور کار:

توضیحات به صورت کامنت در کد ذکر شده است.

```

MAIN.s
1  Stack_Size      EQU      0x00000400
2
3                  AREA     STACK, NOINIT, READWRITE, ALIGN=3
4  Stack_Mem       SPACE    Stack_Size
5  __initial_sp
6
7                  PRESERVE8
8                  THUMB
9
10 ; Vector Table Mapped to Address 0 at Reset
11                  AREA     RESET, DATA, READONLY
12                  EXPORT   __Vectors
13
14 __Vectors        DCD      __initial_sp          ; Top of Stack
15                  DCD      Reset_Handler        ; Reset Handler
16
17                  ALIGN
18
19
20 DATAIN DCB 9
21
22                  AREA MYDATA, DATA, READWRITE
23 COEFS SPACE 36
24 KPSTR SPACE 100
25
26                  AREA     MAIN, CODE, READONLY
27                  ENTRY
28                  EXPORT   Reset_Handler
29
30 Reset_Handler
31 ; *** MAIN STARTS HERE ***
32
33 ; *** REGISTER NAMING ***
34 ROW RN R12
35 COL RN R11
36 FACT_PAR RN R10
37 FACT_ANS RN R9
38 ROW_MINUS_FACT RN R8
39 COL_MINUS_FACT RN R7
40 DIFF_FACT RN R6
41 PASCAL_ANS RN R5
42 ; *****
43
44 LDR R0, =COEFS ; ADDRESS OF COEFS
45 BL KHPA

```

```

44      BL KHPA
45
46      ; *** REGISTER NAMING ***
47      N RN R12
48      INDEX RN R11
49      KPSTR_ADDR RN R10
50      COEFS_ADDR RN R9
51      ; *****
52
53      BL STR_BINOMIAL_EQ
54
55      ENDLESS
56      B ENDLESS ; END OF MAIN
57      ; *****
58
59      ; *** KHPA ROUTINE / GETS R0 ***
60      KHPA PROC
61          PUSH {LR}
62          PUSH {R0}
63          LDR R0, =DATAIN
64          LDR ROW, [R0]
65          POP {R0}
66          MOV COL, #1
67      NEXT_KHPA
68          CMP ROW, COL
69          BLT KHPA_END
70          PUSH {LR, R1, R0, R2}
71          BL PASCAL
72          LDR R1, =COEFS
73          SUB R0, COL, #1
74          MOV R2, #4
75          MUL R0, R2
76          STR PASCAL_ANS, [R1, R0]
77          POP {LR, R1, R0, R2}
78          ADD COL, COL, #1
79          B NEXT_KHPA
80      KHPA_END
81          POP {LR}
82          BX LR ; SAVES DATA IN COEFS
83          ENDP
84      ; *****
85

```

```

MAIN.s
87  PASCAL PROC
88      PUSH {LR}
89      MOV R0, ROW
90      MOV R1, COL
91      SUB R0, R0, #1
92      SUB R1, R1, #1
93      ; --- (ROW-1)! ---
94      MOV FACT_PAR, R0
95      BL FACT
96      MOV ROW_MINUS_FACT, FACT_ANS
97      ; --- (COL-1)! ---
98      MOV FACT_PAR, R1
99      BL FACT
100     MOV COL_MINUS_FACT, FACT_ANS
101     ; --- (ROW-COL)! ---
102     SUB R0, R0, R1
103     MOV FACT_PAR, R0
104     BL FACT
105     MOV DIFF_FACT, FACT_ANS
106     ; -----
107     PUSH {R0, R1}
108     MOV R0, ROW_MINUS_FACT
109     MOV R1, COL_MINUS_FACT
110     BL DIV
111     MOV R1, DIFF_FACT
112     BL DIV
113     MOV PASCAL_ANS, R0
114     POP {R0, R1}
115     POP {LR}
116     BX LR ; RETURN PASCAL_ANS
117     ENDP
118 ; *****
119
120
121 ; *** FACT ROUTINE ***
122 FACT PROC
123     MOV FACT_ANS, #1
124 FACT_LOOP
125     CMP FACT_PAR, #1
126     BLE FACT_END
127     MUL FACT_ANS, FACT_PAR
128     SUB FACT_PAR, FACT_PAR, #1
129     B FACT_LOOP
130 FACT_END
131     BX LR ; RETURN
132     ENDP
133 ; *****

```

```

134
135 ; *** STR_BINOMIAL_EQ ROUTINE ***
136 STR_BINOMIAL_EQ PROC
137     PUSH {LR}
138
139     LDR N, =DATAIN
140     LDR N, [N]
141     SUB N, N, #1
142
143     MOV INDEX, #0
144
145     LDR KPSTR_ADDR, =KPSTR
146     LDR COEFS_ADDR, =COEFS
147 STR_BINOMIAL_EQ_BACK
148     LDR R0, [COEFS_ADDR]
149     ADD COEFS_ADDR, COEFS_ADDR, #4
150     BL BCD_TO_ASCII_AND_STR
151     MOV R0, #' '
152     BL STR_AND_INDEX
153
154     CMP INDEX, N
155     BEQ Y_STR
156
157 X_STR
158     MOV R0, #'X'
159     BL STR_AND_INDEX
160
161     SUB R0, N, INDEX
162     CMP R0, #1
163     BEQ Y_STR
164
165     MOV R0, #'^'
166     BL STR_AND_INDEX
167
168     SUB R0, N, INDEX
169     BL BCD_TO_ASCII_AND_STR
170
171     CMP INDEX, #0
172     BEQ CHECK_END
173
174 Y_STR
175     MOV R0, #'Y'
176     BL STR_AND_INDEX
177
178     CMP INDEX, #1
179     BEQ CHECK_END

```

```

MAIN.s
180
181     MOV R0, #'^'
182     BL STR_AND_INDEX
183
184     MOV R0, INDEX
185     BL BCD_TO_ASCII_AND_STR
186
187 CHECK_END
188     ADD INDEX, INDEX, #1
189     CMP INDEX, N
190     BGT END_STR_BINOMIAL_EQ
191
192 PLUS_STR
193     MOV R0, #' '
194     BL STR_AND_INDEX
195     MOV R0, #'+'
196     BL STR_AND_INDEX
197     MOV R0, #' '
198     BL STR_AND_INDEX
199
200     B STR_BINOMIAL_EQ_BACK
201
202 END_STR_BINOMIAL_EQ
203     POP {LR}
204     BX LR ; RETURN
205     ENDP
206 ; *****
207
208
209 ; *** STR_AND_INDEX ROUTINE ***
210 STR_AND_INDEX PROC
211     STB R0, [KPSTR_ADDR]
212     ADD KPSTR_ADDR, KPSTR_ADDR, #1
213     BX LR ; RETURN
214     ENDP
215 ; *****
216
217
218 ; *** BCD_TO_ASCII_AND_STR ROUTINE ***
219 BCD_TO_ASCII_AND_STR PROC
220     MOV R2, #0
221     PUSH {LR}
222 BCD_TO_ASCII_BACK
223     MOV R1, #10
224     BL DIV
225     ADD R1, R1, #'0'
226

```



```
MAIN.s
226
227     PUSH {R1}
228     ADD R2, R2, #1
229
230     CMP R0, #0
231     BNE BCD_TO_ASCII_BACK
232
233 STR_BACK
234     POP {R0}
235     STRB R0, [KPSTR_ADDR]
236     ADD KPSTR_ADDR, KPSTR_ADDR, #1
237
238     SUBS R2, R2, #1
239     BNE STR_BACK
240
241     POP {LR}
242     BX LR ; RETURN
243     ENDP
244 ; *****
```

```

243     ENDP
244 ; *****
245
246 ; *** DIV ROUTINE R0/R1***
247 DIV PROC
248     ; save R2 and R3 on the stack
249     PUSH {R2, R3}
250     ; check if the divisor is zero
251     CMP R1, #0
252     ; if the divisor is zero
253     BCC DIV_END
254     ; check whether R0 is less than R1
255     CMP R0, R1
256     ; branch if it was
257     BGE DIV_OK
258     ; handle this condition
259     MOV R1, R0
260     MOV R0, #0
261     B DIV_END
262 DIV_OK
263     ; initialize the quotient in R2 and the remainder in R3
264     MOV R2, #0
265     MOV R3, R0
266     ; loop until the remainder is less than the divisor
267 LOOP
268     ; subtract the divisor from the remainder
269     SUB R3, R3, R1
270     ; increment the quotient
271     ADD R2, R2, #1
272     ; compare the remainder and the divisor
273     CMP R3, R1
274     ; if the remainder is greater than or equal to the divisor, repeat the loop
275     BGE LOOP
276     ; store the quotient in R0 and the remainder in R1
277     MOV R0, R2
278     MOV R1, R3
279 DIV_END
280     ; restore R2 and R3 from the stack
281     POP {R2, R3}
282     ; return from the subroutine
283     BX LR ; returns R0 as Quotient & R1 as Remainder
284     ENDP
285 ; *****
286
287
288 ; *** THE END ***
289     END

```

تست:

```

20  DATAIN DCB 9
21
22      AREA MYDATA, DATA, READWRITE
23  COEFS SPACE 36
24  KPSTR SPACE 100

```

```
001 008 028 056 070 056 028 008 001
```

```
1 X^8 + 8 X^7Y + 28 X^6Y^2 + 56 X^5Y^3 + 70 X^4Y^4 + 56 X^3Y^5 + 28 X^2Y^6 + 8 XY^7 + 1 Y^8.
```